

Dstrmaysam Healthcare Knowledge Agent

System Feature Guide

This document details the major features, design choices, operational behavior, and agent workflow of the healthcare knowledge-agent system. It is written for product, technical, and delivery stakeholders who need a clear view of what the platform does and why the current architecture supports those capabilities.

1. Product Overview

The system is an authenticated healthcare knowledge assistant. It combines document question answering, structured deterministic lookup, administrator-controlled document ingestion, patient-detail querying, observability, evaluation metrics, and role-aware governance into one Streamlit and FastAPI application.

- Primary interface: Streamlit multipage frontend for chat, dashboard, patient details, users, and documents.
- Backend API: FastAPI service with authentication, chat orchestration, document APIs, admin APIs, and dashboards.
- Agent runtime: LangGraph-based execution with fast planned paths for common RAG and deterministic questions.
- Model provider: Azure OpenAI through LangChain OpenAI integrations, allowing Azure deployment configuration.
- Retrieval: OpenSearch Serverless in AWS mode and persistent ChromaDB in local mode.
- Observability: Langfuse traces, tool flow metadata, latency breakdowns, token counts, and RAGAS scores.

Why this feature set exists

Healthcare knowledge work often mixes policy interpretation, operational lookup, and document evidence. The system separates those modes intentionally: RAG is used for narrative document content, deterministic lookup is used for exact database facts, and dashboards make the quality and latency of each interaction auditable.

2. Deployment And Runtime Modes

AWS development mode

- Secrets come from AWS Secrets Manager so application credentials, Azure OpenAI keys, and Langfuse keys are not hardcoded.
- Documents are uploaded to S3 and indexed into AWS OpenSearch Serverless.
- OpenSearch index creation is checked before indexing/searching; if the index is absent, the backend creates a compatible knn_vector mapping.
- Chat history can use the configured deployed history backend, including DynamoDB or Postgres depending on configuration.
- ECS task definitions and IAM policies define the deployable container environment.

Local mode

- `LOCAL_TEST_ADMIN_ENABLED=true` switches the backend to local resources.
- Local files are stored under the project data folder and indexed into persistent ChromaDB.
- Secrets are read from `.env/local JSON`; admin-created users persist to `data/local_app_secret.json`.
- Chat history is persisted in Postgres for local mode so browser refreshes and restarts keep conversation history.

Reasoning

The split between AWS mode and local mode lets developers work without depending on cloud services while keeping the deployed path close to production-like AWS architecture. A single switch reduces environment drift.

3. Identity, Roles, And Administration

- Authentication uses bearer tokens issued by the backend after username/password login.
- The login response includes username, roles, departments, and password_change_required.
- First-login or reset-password users must change password before chat, documents, or admin routes are usable.
- Admin APIs create users, update roles/departments, reset passwords, and protect the final admin from losing admin role.
- Known roles include admin, staff, doctor, nurse, pharmacy, clinical_governance, and manager.
- Departments are normalized for consistent authorization and filtering.

Reasoning

Role and department claims are kept close to the auth token so the agent, document filtering, deterministic lookup, and admin APIs can make consistent access decisions. Forced password change protects temporary-password workflows without needing a full identity provider integration.

4. Document Management And Indexing

- Admins can upload PDF, DOCX, TXT, Markdown, and CSV files.
- Non-CSV documents are stored as raw documents and parsed into chunks for RAG.
- CSV uploads are ingested into Postgres deterministic lookup tables and added to the document manifest as metadata-only assets.
- The documents page always shows indexed document table data: file name, key, chunks, category, type, status, and URI.
- Admins can trigger ingestion and indexing from the frontend.
- Admins can delete all search/vector indexes after confirming with their admin password; uploaded source files and deterministic lookup rows are preserved.
- Incremental indexing skips unchanged files, reindexes changed files, deletes removed files, and forces reindex when the configured search index changes.

Reasoning

The ingestion workflow favors repeatability and operational safety. It avoids reprocessing unchanged content, keeps CSV deterministic data separate from vector retrieval, and gives administrators a controlled reset path for search indexes.

5. Retrieval And Knowledge Search

Catalog-guided RAG

- The document catalog loads manifest metadata and matches query terms against title, key, content type, and metadata JSON.
- For rag_search, document_search, and policy_search, the catalog narrows candidates before vector/keyword search.
- Candidate keys are limited to reduce search scope and latency.
- Policy search prefers clinical policy, admin policy, compliance, policy, SOP, pathway, and guideline documents.

- If no catalog candidates match, the system falls back to broad retrieval to avoid false negatives.

OpenSearch and ChromaDB

- AWS mode uses OpenSearch Serverless with vector plus keyword search.
- Local mode uses persistent ChromaDB with equivalent retrieval contract.
- OpenSearch search uses multi-search where available for vector and keyword search.
- Neighbor chunk expansion fetches adjacent chunks to improve answer completeness.
- Query embedding and retrieval result caches reduce repeated-query latency.

Reasoning

Pure vector search can miss metadata-specific questions, while pure keyword search misses semantic matches. Combining catalog narrowing, vector search, keyword search, access filtering, and neighbor expansion improves precision and context coverage.

6. Deterministic Lookup And Patient Data

- The Postgres deterministic lookup tool handles exact facts such as doctors on call, patients, appointments, wards, contacts, departments, formularies, and schedule rows.
- Uploaded CSV lookup files are stored in Postgres as the source of truth.
- CSV files are represented in the document manifest as metadata-only assets so administrators can see that they are available.
- The patient details dashboard filters and queries configured Postgres tables, including patient and appointment-style data.
- Deterministic answers preserve exact values and return structured rows instead of relying on vector similarity.

Reasoning

Structured healthcare data should not be approximated through embeddings. Postgres lookup provides precise results, supports filters, and keeps table data queryable without inflating the vector index.

7. Chat Experience

- The Streamlit app uses native multipage navigation for smooth page switching.
- Chat has a scrollable message window with a fixed query form below the chat area.
- The user query appears in the chat window immediately while the answer is processing.
- Chat sessions are persisted and available from the Chat page sidebar only, reducing non-chat page overhead.
- Responses include citations/sources from retrieval-backed tools and optional snippets.
- The frontend hides operational response details from the normal chat view; those details are available in dashboards and traces.

Reasoning

The chat page is optimized for day-to-day use. Heavy admin pages and history loading are isolated so typing and chat interaction remain responsive.

8. Observability, Evaluation, And Dashboarding

- Each chat request receives a trace ID; Langfuse traces are used when configured and local IDs are used as fallback.
- Trace metadata includes user/session, model, prompt label, tools used, tool flow,

sources, safety, token counts, and latency.

- The admin dashboard shows per-query and aggregate views for latency, tokens, users, tools, models, sources, guardrails, and RAGAS.
- Latency breakdown includes history, prompt, trace setup, LLM setup, LLM calls, retrieval, catalog, index checks, guardrail, safety, and history save timings.
- RAGAS scoring runs in the background and publishes faithfulness, answer relevancy, context precision, and context recall to Langfuse.
- The eval script can publish RAGAS scores to response traces and create an evaluation-run summary trace.

Reasoning

Healthcare-facing knowledge systems need auditability. The dashboard and Langfuse traces make it possible to diagnose latency, tool choice, retrieval quality, prompt behavior, and answer quality after the fact.

9. Safety And Guardrails

- PHI redaction is applied before agent input.
- Initial safety context is included before answer generation.
- Final safety assessment runs after answer generation using actual sources.
- Response-style guardrail rewrites are triggered when the query or draft answer contains risky tone/style markers.
- Guardrails require professional, neutral, concise responses and remove jokes, sarcasm, roleplay, slang, emojis, and theatrical wording.
- The guardrail rewrite uses the fast Azure OpenAI deployment when available.

Reasoning

The system balances latency and safety by avoiding a guardrail call for every answer. It uses deterministic risk detection to decide when an LLM rewrite is needed, then records the outcome in trace metadata.

10. Agent Workflow

The agent uses LangGraph-compatible execution with optimized planned paths. LangChain remains useful for Azure OpenAI model integrations, embeddings, message compatibility, and callback plumbing.

End-to-end chat workflow

1. Request enters /chat with bearer token and optional session_id.
2. Auth dependency validates token and blocks access if password_change_required is true.
3. Query is redacted for PHI and conversation history is loaded.
4. Langfuse trace is opened when configured; otherwise a local trace ID is used.
5. System prompt is loaded from Langfuse or fallback prompt, then response-style baseline is appended.
6. Initial safety assessment is generated and included in agent context.
7. The planner decides whether the query is RAG-only, deterministic-only, multipart, or ambiguous.
8. Fast planned paths execute direct tool calls for clear RAG/deterministic questions.
9. Ambiguous questions fall back to LangGraph tool-calling loop with a maximum LLM-call limit.
10. Tool outputs are bounded into context and one final answer LLM call synthesizes the response.
11. Response guardrail rewrite runs only when tone/style risk is detected.

- 12. Final safety assessment, audit metadata, token estimates, latency breakdown, sources, and tool flow are stored.
- 13. Chat history is persisted and RAGAS/Langfuse enrichment runs in the background.

Workflow diagram

```
User -> FastAPI /chat -> Auth + password gate
      -> PHI redaction -> history load -> Langfuse trace/prompt
      -> safety context -> planner
      -> [fast RAG | fast deterministic | multipart | LangGraph fallback]
      -> tools -> bounded context -> Azure OpenAI answer
      -> optional guardrail rewrite -> final safety
      -> response + sources -> history + Langfuse + background RAGAS
```

RAG document search tool workflow

- Catalog candidate selection: manifest metadata is filtered by role and matched against query terms.
- Search narrowing: candidate document keys are passed into OpenSearch/Chroma retrieval.
- Retrieval: vector and keyword search run against selected documents; broad fallback is used when catalog matching fails.
- Neighbor expansion: adjacent chunks are retrieved to improve context continuity.
- Source packaging: snippets, title, URI, score, metadata, and document keys are returned for citations and RAGAS.

Deterministic lookup workflow

- The planner detects structured-fact intent such as rota, patient, appointment, department, contact, or formulary queries.
- The Postgres lookup service applies user access context and queries matching tables.
- Rows are returned as exact structured JSON.
- The answer synthesizer preserves exact values and does not reinterpret deterministic facts.

Document catalog workflow

- The document catalog tool exposes governed document metadata to the agent when explicitly called.
- Internally, the same catalog logic acts as a helper for RAG narrowing.
- Internal catalog assistance is shown in tool_flow but is not added to tools_used unless the LLM explicitly selected document_catalog.
- This keeps user-visible tool usage honest while still making helper behavior auditable.

Loop limit and fallback

The LangGraph fallback loop caps LLM tool-calling iterations. If the limit is reached, the agent produces a final no-tool answer from accumulated context instead of hanging.

11. Performance And Latency Controls

- Fast planned execution bypasses tool-selection LLM calls for clear query types.
- Fast model routing can use AZURE_OPENAI_FAST_DEPLOYMENT for answer synthesis and guardrail rewrites.
- OpenSearch multi-search combines vector and keyword queries when available.
- Embedding cache and retrieval cache reduce repeated work.
- Catalog candidate cache reduces repeated manifest matching.

- Context budgets keep answer prompts bounded.
- Startup warmup primes LLM clients, Langfuse prompt cache, document manifest, and retrieval path.
- Background history save, trace enrichment, and RAGAS scoring avoid adding synchronous chat latency.

Reasoning

The system optimizes full-answer completion time without removing functionality. It keeps traceability and evaluation while moving nonessential enrichment off the critical path.

12. Main API And UI Features

Backend endpoints

- GET /health: service status, settings summary, registered tools, and warmup status.
- POST /auth/login, GET /auth/me, POST /auth/change-password: authentication and password management.
- POST /chat: main conversational API with sources, tools, trace ID, tokens, latency, safety, and performance metadata.
- GET /chat/sessions and GET /chat/sessions/{session_id}: persisted chat history.
- GET /documents: role-filtered indexed document list.
- Admin users: GET/POST/PATCH /admin/users and POST /admin/users/{username}/reset-password.
- Admin documents: upload, ingest, delete indexes.
- Admin dashboard: per-query and aggregate operational metrics.
- Admin warmup: manual warmup trigger.
- Admin patient details: filtered Postgres patient/appointment data access.

Frontend pages

- Chat: conversational Q&A with session history.
- Dashboard: traces, tool flow, latency, tokens, RAGAS, guardrails, and per-query drilldowns.
- Patient Details: filterable table view over configured Postgres patient data.
- Users: admin user creation, role assignment, department assignment, and password reset.
- Documents: upload, ingest/index, index reset, and indexed-document table.

13. Summary

The system is a role-aware healthcare knowledge platform with a practical split between semantic document retrieval and exact deterministic lookup. LangGraph handles agent workflow, Azure OpenAI provides model and embedding services, OpenSearch/Chroma provide vector search depending on runtime mode, Postgres stores structured facts and local history, and Langfuse/RAGAS make behavior measurable. The design favors auditability, local-developer productivity, and operational control while keeping the chat experience focused and responsive.